E2E ML PROJECT

Predicting Used Car Auction Prices

A full-stack machine learning pipeline for automotive price prediction — from raw auction data to a deployment-ready model.

Dataset: Kaggle — Vehicle Sales Data · Task: Regression · 2026

1. Problem Statement

Used vehicle prices at auction depend on a complex mix of factors — model year, mileage, condition score, body style, transmission, geography, and seasonality. Manual pricing is inconsistent and error-prone at scale.

The goal of this project is to build a machine learning model that learns pricing patterns automatically from historical sales data, enabling accurate and repeatable price estimates across a diverse vehicle inventory.

2. Dataset

Source: Kaggle — Vehicle Sales Data (team sample_7). The working dataset contains 49,998 records and 15 columns, split into 39,998 training rows and 10,000 test rows.

- year, make, model, trim — vehicle identity and age
- body, transmission — configuration signals
- odometer, condition — physical state indicators
- color, interior — cosmetic attributes
- state — geographic context
- sale_year, sale_month, sale_day_of_week — temporal features extracted from saledate
- sellingprice — prediction target (continuous, USD)

3. Methodology

Step 1 — Exploratory Data Analysis

We began with a thorough EDA to understand the data before modeling. Key findings:

- The target variable (sellingprice) shows a strong right skew — addressed with a log transformation
- Year, odometer, and condition are the most correlated numerical features with price
- Geographic variation is significant: average prices differ notably across US states
- Seasonal patterns exist in auction pricing with monthly fluctuations

- Top brands by volume: Ford, Chevrolet, Nissan, Toyota dominate the auction sample

Step 2 — Missing Value Imputation

- odometer and condition: filled with median per model year group
- Categorical features (transmission, body, color, interior, trim): filled with mode per make/model group
- Rows missing sellingprice or saledate were dropped as structurally unusable
- Any remaining blanks after group imputation were tagged as 'Unknown'

Step 3 — Feature Engineering & Preprocessing

- Date parsing: saledate cleaned; sale_year, sale_month, sale_day_of_week extracted as temporal features
- Categorical encoding: OrdinalEncoder with handle_unknown=use_encoded_value
- Train/test split: 80/20 (39,998 train / 10,000 test), random_state=42
- Processed dataset saved to data/processed/

Step 4 — Model Training & Hyperparameter Tuning

Two model families were trained with GridSearchCV (4-fold CV, scoring on negative MAE):

- Random Forest — 12 candidates, 48 fits. Best: n_estimators=200, max_depth=None, min_samples_leaf=1
- Gradient Boosting — 8 candidates, 32 fits. Best: n_estimators=200, max_depth=5, learning_rate=0.1

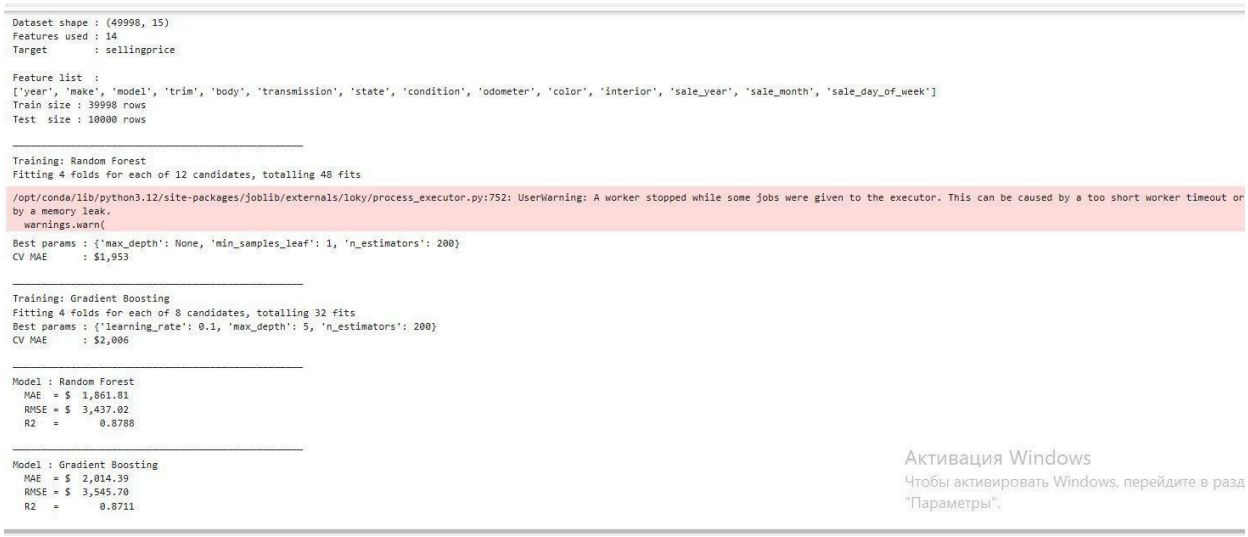

Fig. 1 — Training output: GridSearchCV results and test set metrics for both models

Step 5 — Production Readiness

The best model was validated with a Giskard automated scan (report saved to reports/giskard_scan_report.html) and benchmarked for inference speed comparing joblib vs pickle. joblib was selected as the production serialization format based on comparable file size, though pickle demonstrated faster load+predict speed in this benchmark.

4. Results

Model Performance on Test Set

Metric	Random Forest ★	Gradient Boosting
MAE	$1,861.81	$2,014.39
RMSE	$3,437.02	$3,545.70
R²	0.8788	0.8711
CV MAE (train)	$1,953	$2,006

Random Forest was selected as the best model based on lowest MAE ($1,861.81). It explains 87.9% of variance in auction selling prices (R² = 0.8788). On average, predictions deviate by ~$1,862 from actual prices.

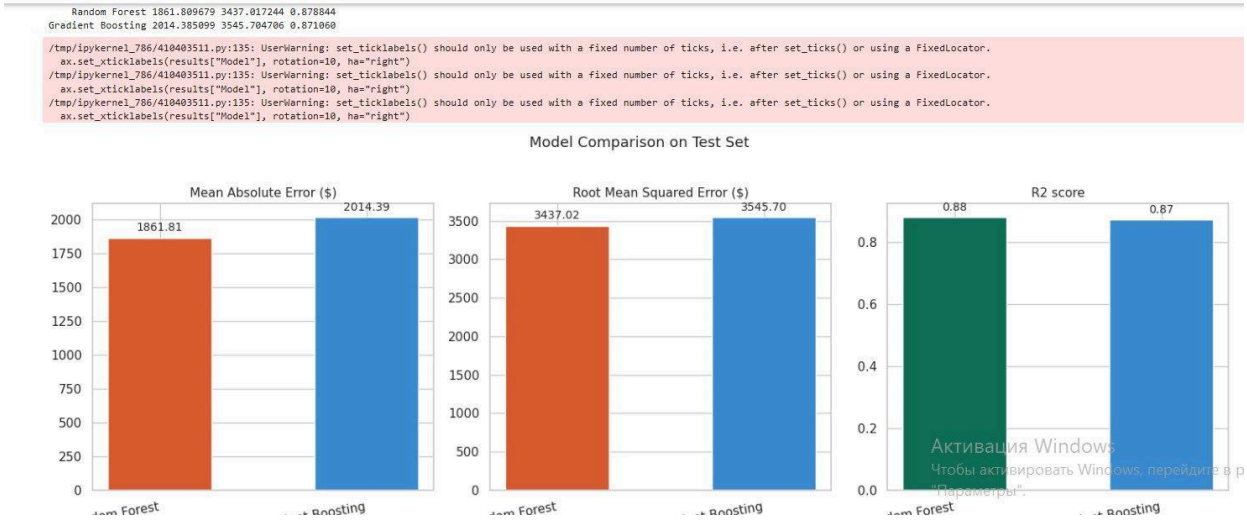

Fig. 2 — Model comparison: MAE, RMSE, and R² on test set

Actual vs Predicted Prices

Both models track the diagonal closely across the full price range ($0–$140,000), confirming good generalization. Random Forest shows tighter clustering around the perfect-prediction line.

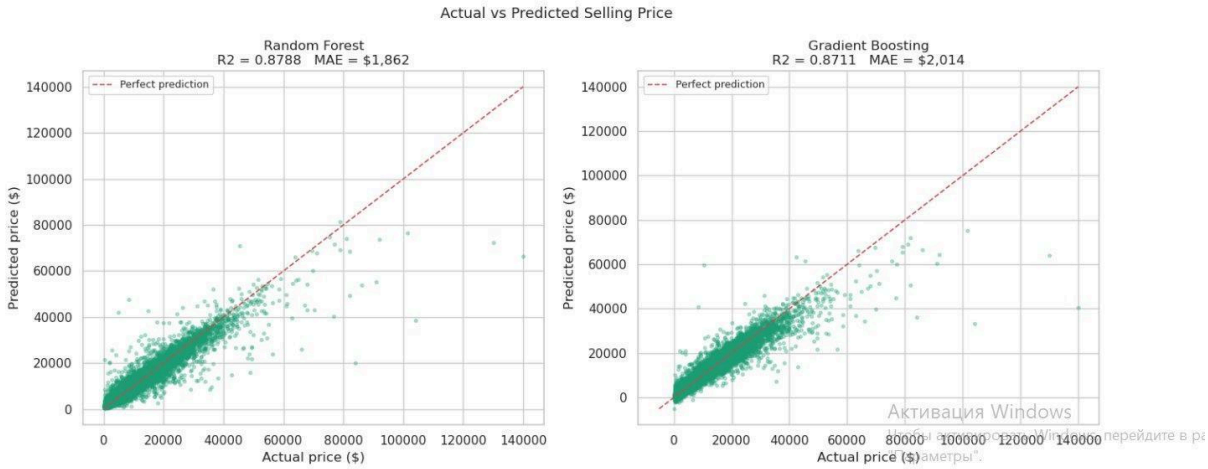

Fig. 3 — Actual vs Predicted selling price: Random Forest (left) and Gradient Boosting (right)

Residual Analysis

Residuals are centered around zero for both models with no clear systematic bias, indicating well-calibrated predictions. Both models show increased variance at higher price points (heteroscedasticity), which is expected for used car pricing at the luxury end.

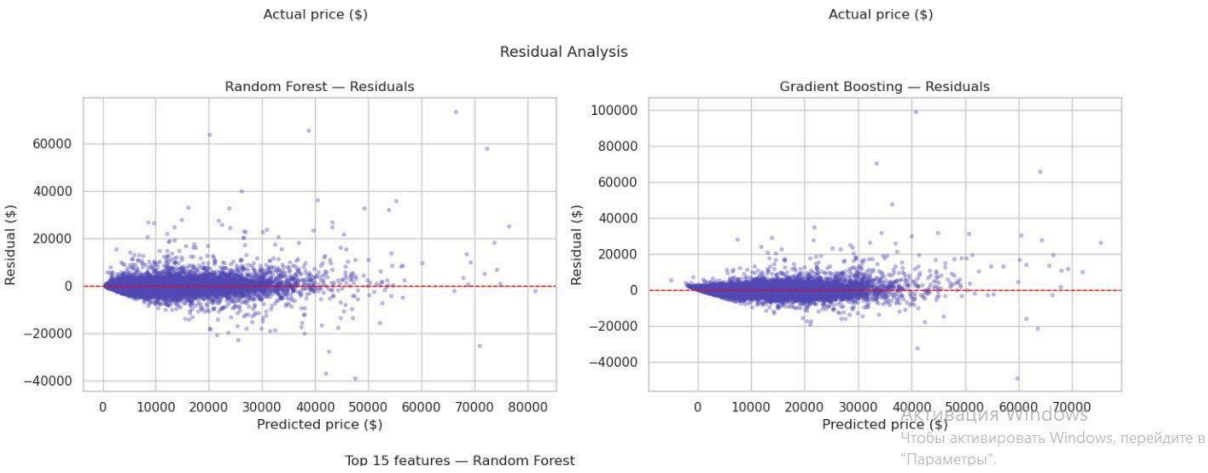

Fig. 4 — Residual analysis: Random Forest (left) and Gradient Boosting (right)

Feature Importance

MDI feature importance from the best model (Random Forest) shows odometer as the dominant predictor by a large margin (~0.39), followed by body style, make, year, trim, and model. Temporal features (sale_month, sale_day_of_week, sale_year) and cosmetic attributes (color, interior) have near-zero importance.

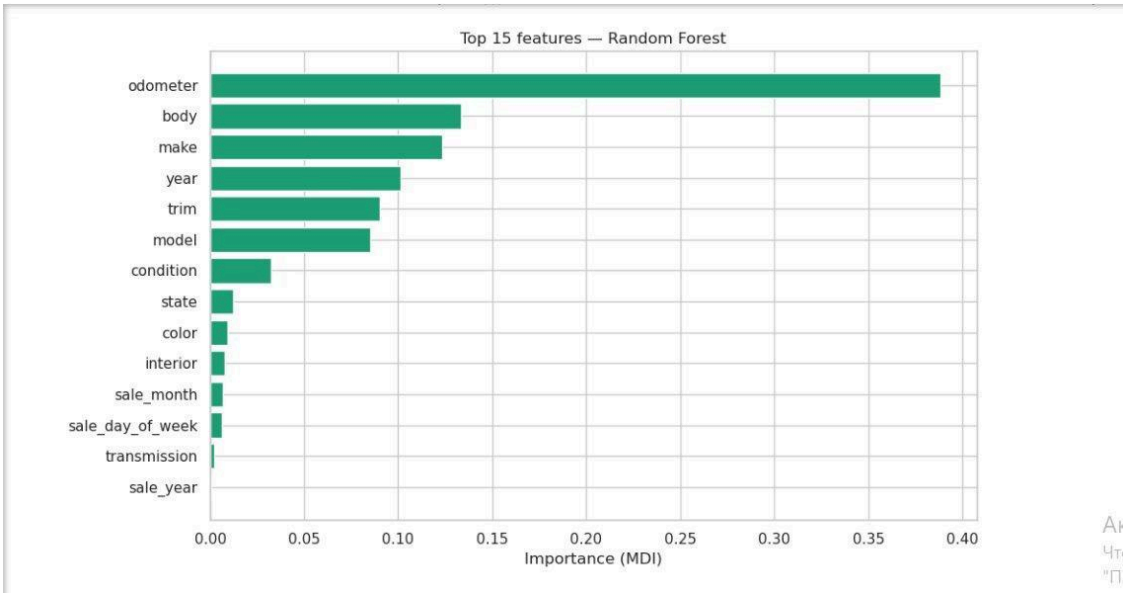

Fig. 5 — Top 15 features by MDI importance (Random Forest)

Best Model — Sanity Check

After saving and reloading best_model.joblib, predictions on 5 test samples match exactly, confirming lossless serialization:

```
Best model: Random Forest
  MAE  = $1,861.81
  RMSE = $3,437.02
  R2   = 0.8788

Saved to: ../models/best_model.joblib
Comparison table saved.
Sanity check — first 5 predictions:
  Original : [ 5708. 27258.  8565. 22958. 23542.]
  Loaded   : [ 5708. 27258.  8565. 22958. 23542.]
Model loads and predicts correctly.
```

Fig. 6 — Sanity check: original vs loaded model predictions are identical

5. Tech Stack

Language	Python 3.12
Data processing	pandas, numpy
Visualization	matplotlib, seaborn, plotly

ML framework	scikit-learn
Model evaluation	Giskard
Serialization	joblib, pickle
Notebook env	Jupyter
Version control	Git / GitHub (private repo)

6. Repository Structure

- data/ — raw sample (sample_7.csv) and processed dataset
- notebooks/ — EDA.ipynb, DataPreparation.ipynb, ModelTraining.ipynb, DeploymentPrep.ipynb
- reports/ — giskard_scan_report.html, model comparison outputs
- models/ — best_model.joblib, model_comparison.csv
- backend/ — reserved for API/serving layer
- frontend/ — reserved for UI layer
- requirements.txt, README.md, .gitignore — project config files